

Privacy Audit: Membership Inference Attacks on Generative Models and the Role of Differential Privacy

Sabrina Haniff
University of Guelph

May 2026

Abstract

Generative models trained on sensitive data are vulnerable to membership inference attacks, where an attacker can determine whether a specific sample was part of the training set by exploiting the model’s tendency to memorize. We trained a GAN on MNIST and demonstrated that a threshold-based membership inference attack achieves 51.93% accuracy against the undefended discriminator, a modest but measurable advantage over the 50% random baseline. We then applied Differential Privacy via DP-SGD and observed that attack accuracy dropped to 49.92%, with the confidence gap between members and non-members collapsing from 0.0290 to -0.0001. These results suggest that DP-SGD can effectively reduce this specific class of membership inference vulnerability in GAN discriminators, though the tradeoff in model utility is significant.

1 Introduction

Generative AI models are increasingly being trained on sensitive data including medical records, personal images, and private communications. A side effect of this training is that models tend to memorize parts of the data they were trained on. This creates a real privacy risk: if an attacker can query the model, they may be able to figure out whether a specific person’s data was used in training. This is called a membership inference attack.

This is not a theoretical concern. In a well-known demonstration, researchers prompted GPT-2 with repeated text and were able to extract verbatim training data including personal details like names and credentials. The same underlying vulnerability exists in generative models more broadly.

This project focuses on that vulnerability in the context of GANs, specifically targeting the discriminator. The discriminator trains directly on real data, which makes it the main source of leakage. We measure how much information leaks through a simple membership inference attack, and then test whether Differential Privacy can close that gap.

2 Background

2.1 Generative Adversarial Networks

A GAN consists of two neural networks: a generator and a discriminator. The generator takes random noise as input and learns to produce fake samples that resemble the training data. The

discriminator sees both real and fake samples and learns to tell them apart. They train together in a loop where the generator gets better at fooling the discriminator, and the discriminator gets better at catching fakes.

The privacy issue comes from the discriminator. Because it trains directly on all the real samples, it memorizes what they look like. After training, it tends to assign higher confidence scores to samples it has seen before compared to ones it has not. That difference is what an attacker exploits.

2.2 Membership Inference Attacks

A membership inference attack tries to determine whether a specific sample was in the training set. The core idea is that models tend to be more confident on data they have already seen. For a GAN discriminator, training samples tend to receive higher confidence scores than unseen samples. An attacker exploits this by querying the model and checking whether the score falls above or below a threshold.

Attack accuracy above 50% means the attack is extracting real signal. Exactly 50% means the attacker is no better than random guessing, which is the target we want DP to push us toward. It is worth noting that even a small advantage above 50% is meaningful in practice. At scale, a 1-2% statistical edge translates to correctly identifying a large number of individuals.

2.3 Differential Privacy

Differential Privacy is a mathematical framework for limiting how much any single training sample can influence a model’s output. The formal guarantee is that adding or removing one person’s data from the training set should not significantly change what the model produces. This is captured by the privacy budget ϵ : smaller values mean stronger protection but typically degrade model performance.

In practice, DP is applied to neural network training through DP-SGD [4]. It works in two steps during each training iteration: first, per-sample gradients are clipped so no single example can dominate the update, and second, calibrated Gaussian noise is added before the optimizer applies the update. The result is a model that learns general patterns without memorizing specific examples.

3 Methodology

3.1 Dataset and Experimental Setup

We used the MNIST handwritten digit dataset. We split it into two groups of 5,000 images: a member set used for training and a non-member set held out entirely for evaluation. This split is established before training so we have clean ground truth labels when measuring attack accuracy. All experiments were run using PyTorch 2.11 on an Apple M-series processor using the MPS backend. Training the baseline GAN took approximately 2-3 minutes for 50 epochs. The DP-GAN took noticeably longer due to the per-sample gradient clipping overhead introduced by Opacus, roughly 3-4x slower per epoch.

3.2 Baseline GAN Training

We trained a fully connected GAN with a 64-dimensional latent space. The generator maps noise vectors to 28x28 images through four linear layers with LeakyReLU activations and batch normal-

ization. The discriminator classifies flattened 784-dimensional images through three linear layers with LeakyReLU and dropout. Both networks were trained for 50 epochs using the Adam optimizer with learning rate 0.0002 and batch size 64. These hyperparameters are standard for small GAN experiments on MNIST and produced stable training without mode collapse.

3.3 Membership Inference Attack

We ran a threshold-based membership inference attack against the trained discriminator. For each sample in both the member and non-member sets, we queried the discriminator and recorded its confidence score. Samples scoring above 0.5 were classified as members and samples scoring below 0.5 were classified as non-members. We report average member confidence, average non-member confidence, the gap between them, and overall attack accuracy.

3.4 DP-SGD Defense

We retrained the GAN with Differential Privacy applied to the discriminator using the Opacus library [5]. We chose $\epsilon = 10$ as a starting point because it represents a relatively loose privacy budget, which lets the model still attempt to learn while providing a measurable defense. δ was set to 10^{-5} and the maximum gradient norm to 1.0. DP was applied only to the discriminator since the generator never directly sees training data and is therefore not the primary leak source.

One implementation challenge worth noting: applying DP to the discriminator while still training the generator required a custom setup. Because Opacus wraps the discriminator’s optimizer and hooks into every backward pass, any gradient flow through the discriminator during the generator step triggered errors. We resolved this by training the generator against a shadow copy of the discriminator weights that has no Opacus hooks attached. After each discriminator update, the shadow copy is synced to the current DP discriminator weights before the generator step runs.

4 Results

4.1 Baseline Privacy Leakage

Without any defense, the discriminator assigned an average confidence of 0.7057 to member samples and 0.6767 to non-member samples, producing a gap of 0.0290. The membership inference attack achieved 51.93% accuracy. The gap is modest, which reflects the fact that MNIST is a simple dataset and the GAN generalized reasonably well. However, the attack still clears the random baseline, confirming that the discriminator’s memorization is detectable. On a more complex dataset with more sensitive data, this gap would likely be larger.

4.2 DP Defense Effectiveness

After applying DP-SGD with $\epsilon = 10$, average confidence for members dropped to 0.5227 and for non-members to 0.5228, producing a gap of -0.0001. Attack accuracy fell to 49.92%, just below the random baseline. The confidence distributions for members and non-members became nearly identical, collapsing into a narrow range around 0.52. For this specific attack on this dataset, the leakage signal was effectively eliminated. It is worth noting that 49.92% is statistically indistinguishable from a coin flip where the attacker gains essentially no advantage over random guessing.

Model	Member	Non-member	Gap	Attack Acc.
GAN (no DP)	0.7057	0.6767	0.0290	51.93%
DP-GAN ($\epsilon=10$)	0.5227	0.5228	-0.0001	49.92%

Table 1: Membership inference attack results for baseline and DP-GAN.

4.3 Privacy-Utility Tradeoff

The privacy gains came at a real cost. In the baseline GAN, discriminator loss decreased from 1.39 to 0.81 over 50 epochs, showing that it learned effectively. In the DP-GAN, discriminator loss barely moved, staying around 1.38 throughout all 50 epochs. The noise from DP-SGD essentially prevented the discriminator from learning much at all. This is the core tension in differentially private training: the same noise that protects privacy also gets in the way of learning.

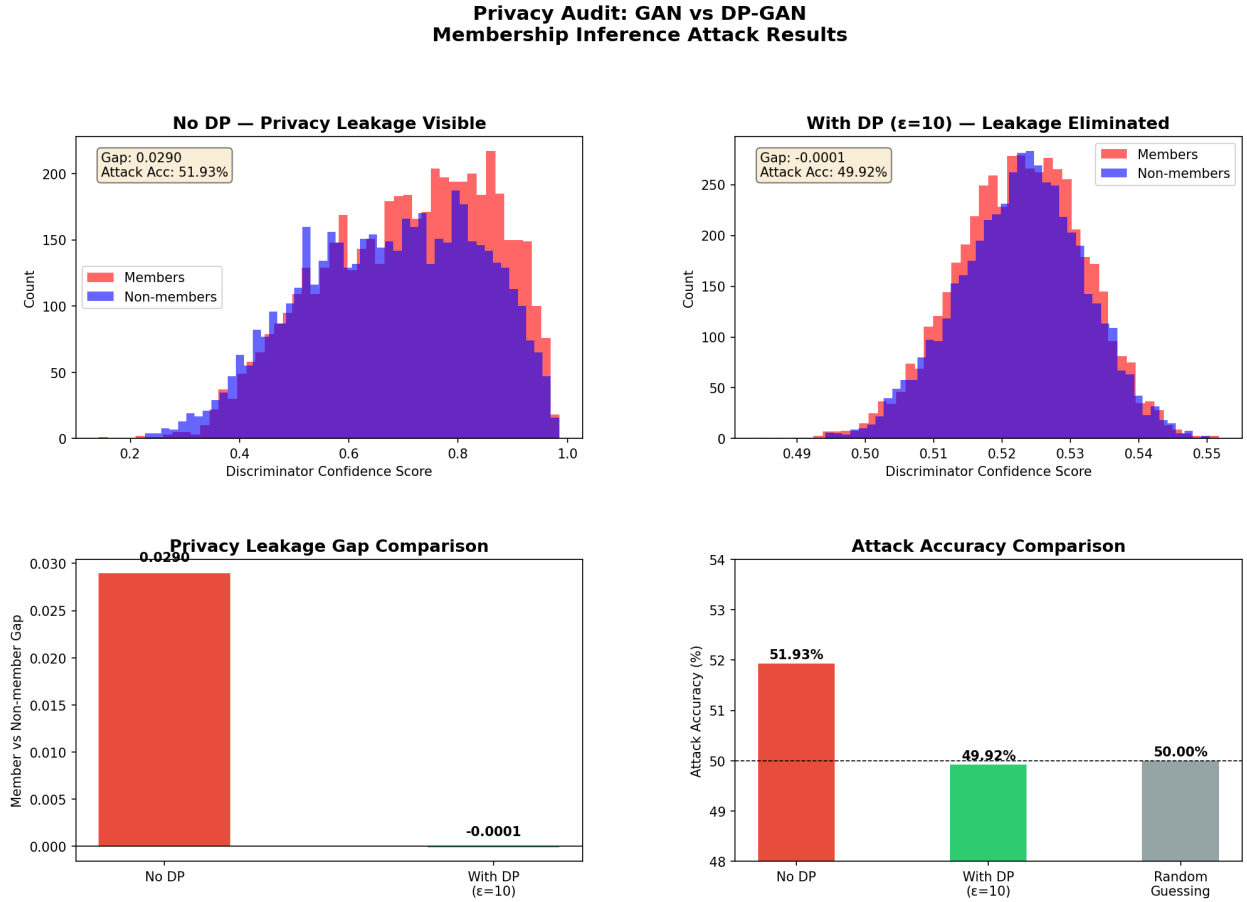


Figure 1: Side-by-side comparison of membership inference attack results for baseline GAN and DP-GAN. Top row shows confidence score distributions. Bottom row shows gap and attack accuracy comparisons.

5 Discussion

The takeaway here is twofold. First, even a model that is not obviously overfitting still leaks information about its training data. The baseline GAN generalized reasonably on MNIST, yet the attack still worked. This suggests that regularization and careful training alone are not enough to protect privacy in sensitive settings. You need a mathematical guarantee.

Second, DP works, but it is not a free lunch. At $\varepsilon = 10$, the attack was reduced to chance-level performance on this MNIST setup. But the discriminator essentially stopped learning. In a real deployment where the generative model needs to actually produce useful outputs, this level of noise would be a serious problem. Finding a tighter ε that still allows learning is an open question, and one that researchers like Dwork [3] and others are actively working on.

It is also worth being honest about the limitations of this experiment. MNIST is a simple, low-stakes dataset. The baseline attack accuracy of 51.93% is weak, which partly reflects how well the model generalized rather than a genuine absence of leakage. On a more complex dataset with higher-dimensional, more sensitive data, we would expect both the baseline leakage and the cost of DP to be more pronounced.

6 Conclusion

We built a system to audit privacy leakage in a GAN discriminator using membership inference attacks, and tested whether Differential Privacy could serve as a defense. The results showed that even a reasonably trained GAN leaks information, and that DP-SGD reduced that leakage to effectively zero for this specific attack and dataset. The gap collapsed from 0.0290 to -0.0001 and attack accuracy dropped from 51.93% to 49.92%.

The broader point is that as generative AI gets deployed in sensitive domains, the question of what these models memorize and leak is not optional. This project is a small-scale demonstration of a real problem. Next steps would include testing across different values of ε , evaluating on more complex datasets, and extending the attack to target the generator directly, since recent work has shown generators can also leak training data under certain conditions.

References

- [1] Goodfellow, I., et al. (2014). Generative adversarial nets. *Advances in Neural Information Processing Systems*, 27.
- [2] Shokri, R., et al. (2017). Membership inference attacks against machine learning models. *IEEE Symposium on Security and Privacy*.
- [3] Dwork, C. (2006). Differential privacy. *International Colloquium on Automata, Languages and Programming*.
- [4] Abadi, M., et al. (2016). Deep learning with differential privacy. *ACM SIGSAC Conference on Computer and Communications Security*.
- [5] Yousefpour, A., et al. (2021). Opacus: User-friendly differential privacy library in PyTorch. *arXiv preprint arXiv:2109.12298*.